

Actuate — AWS Storage, Database & Serving Architecture

Where data lives, how it's indexed, and how the server runs

Companion to the Master Implementation Spec. Target: AWS. Spec only — nothing built.

This fills the gap the pipeline spec left open: physical storage, the metadata/index database, the batch + serving compute, and — critically — the **consent/PII boundary enforced at the infrastructure level, not just in code**. Design bias: start pragmatic (you have credits, you want to build), scale later. Every choice below is CPU-friendly to stand up first; GPU compute is isolated to the stages that need it.

1. Data reality (what we're storing, and why it drives the design)

Data class	Shape	Size driver	Store
Raw capture	Multi-cam video (up to 2160 ² @60fps stereo), IMU, glove/exoskeleton encoders, tactile	Video dominates cost	S3 (cold-tiered)
Dense per-frame	Depth maps, masks, MANO, object poses, SLAM traj	Many small arrays/episode	S3 as Zarr (chunked, partial-read)
Canonical (frozen schema)	Parquet (episode/meta) + Zarr (dense) + chunked-MP4 refs	Medium	S3
Delivered datasets	LeRobot v3 (chunked Parquet + MP4 + metadata), RLDS/TFDS	Medium, versioned	S3 (delivery bucket)
Index / catalog	episodes, tasks, embodiments, scenes, consent, quality, provenance, jobs, versions	Small, highly queried	Postgres

The two facts that shape everything: **video is the dominant storage cost** (→ lifecycle tiering, never duplicate video, reference chunked MP4 rather than re-encode), and **the query surface is relational** ("filter by rig, task, embodiment, scene, consent status, quality") (→ a Postgres catalog pointing at S3 blobs, EgoDB-style).

2. Storage — S3 with a consent boundary

Four buckets, separated so the consent/PII gate is an **infrastructure boundary**, not just app logic. All buckets: Block Public Access ON, SSE-KMS encryption, TLS-only bucket policy, access logging, **raw** and **delivery** versioned.

```
s3://actuate-raw-<env>/      # raw uploads. RESTRICTED. Lifecycle → Glacier Deep
Archive after N days.
  <rig>/<capture_id>/...    # No path from here to customers without passing certify.
s3://actuate-work-<env>/    # intermediate + canonical artifacts (Zarr/Parquet/MP4 refs).
Intelligent-Tiering.
  canonical/<episode_id>/vN/...
s3://actuate-delivery-<env>/ # ONLY consent-passed, PII-redacted, packaged datasets.
Versioned.
  <customer>/<dataset>/<version>/...
s3://actuate-artifacts-<env>/ # model checkpoints, sim assets, embodiment URDFs,
reference-hand model.
```

Consent boundary (the safety-critical part). Writes to **actuate-delivery-*** are permitted **only** for the packaging role, and the packaging job refuses to run on any episode whose **consent/pii_status** is not **passed** in the catalog (fail-closed, mirroring the built consent gate). Belt-and-suspenders: the delivery bucket's policy denies writes from any principal except the packaging role, and the raw/work roles have **no PutObject** on delivery. So a bug in app code cannot leak un-consented data into the customer-facing bucket — IAM stops it too.

Access patterns: Zarr/Parquet read directly from S3 via **fsspec/s3fs** (chunked, no full download); dense per-frame reads never pull whole episodes. Delivered datasets handed to customers via **presigned URLs** or customer-scoped IAM, never public.

Cost controls: raw → Glacier Deep Archive lifecycle; work → S3 Intelligent-Tiering; no video re-encode in canonical (reference the source chunks); dataset versioning only on delivery/raw (not on churning intermediate).

3. Database — Postgres catalog + Athena for analytics

Amazon RDS for PostgreSQL (or **Aurora Serverless v2 Postgres** to idle cheaply on credits) is the transactional catalog and job ledger — the EgoDB equivalent.

Core tables (SQLAlchemy models mirror the canonical schema's episode/metadata fields; Alembic migrations):

rigs, embodiments, scenes, demonstrators, tasks
 episodes (→ S3 URIs, rig, scene, demonstrator, task, schema_version, tier, effective_hours)
 consent (capture_id, status ENUM, fail-closed; gates delivery)
 certifications (episode_id, quality 1-5, speed, mistakes[], strategy_alignment, pii_status)
 annotations (episode_id, paraphrases[], subtasks[], subgoal_frames[])
 retarget_results (episode_id, embodiment, joint/ee refs, sim_validation status)
 datasets (customer, name, version, tier, manifest URI)
 jobs (id, stage, episode_id, status, provenance, checkpoint URI, logs ref)

- **Postgres** = source of truth for status, consent, provenance, and the "give me episodes where task=X and embodiment=Y and consent=passed and quality≥4" queries the dashboard and packaging need.
- **Athena** (serverless SQL over the canonical Parquet in S3) = heavy **diversity/composition analytics** (scene-vs-demonstrator diversity per EgoVerse, effective-hours rollups per EgoScale) without loading data or hammering RDS. Read-only, glued via a Glue catalog.

Rule: **blobs in S3, everything queryable in Postgres with an S3 URI pointer**. No large arrays in the DB.

4. Compute — batch pipeline + serving

4.1 Batch processing (the pipeline runs here)

Containerize the **actuate** package (one Docker image, GPU + CPU variants via extras). Each pipeline stage is a job; the per-episode DAG is orchestrated so failures are resumable and every stage checkpoints.

- **Orchestration: AWS Step Functions** state machine for the DAG: **ingest** → **perceive(hands/depth/objects/pose/slam)** → **fuse** → **canonical** → **certify** → **retarget** → **package**. Each state invokes an AWS Batch job and writes status to **jobs** in Postgres. Fan-out per episode; the consent-gate state short-circuits to "quarantine" on failure.
- **GPU jobs** → **AWS Batch on GPU EC2** (g5/g6, **Spot** for cost): L1 perception, L5 retargeting training + sim validation, L4/L6 VLM/LLM-judge.
- **CPU jobs** → **AWS Batch (Fargate or EC2 CPU)**: L0 (partial), L2 fusion, L3 canonical, L4 non-VLM, L7 exporters.
- **Ingest trigger**: upload to **actuate-raw** → S3 event → EventBridge → Step Functions start (or manual **actuate** CLI kick for dev).

Start-simple option for day one: skip Step Functions, run the **actuate run all** CLI on a single **GPU EC2** box (g5.xlarge, Spot) against S3 + RDS, dispatched via a small **SQS** queue + worker. Graduate to Batch + Step Functions when volume warrants. The CLI/API-first

design means the same library code runs locally, on one EC2 box, or under Batch — no rewrite.

4.2 Serving (the API + dashboard)

- **FastAPI service** → **ECS Fargate** behind an ALB (or **App Runner** for less ops). It reads RDS + S3, dispatches pipeline runs (Step Functions/Batch), returns job status and dataset manifests. It is a **consumer of the `actuate` library**, never a dependency of it.
- **Web dashboard** → **S3 + CloudFront** static hosting, calls the API. (EgoDB's web viewer is the reference for the query/browse surface.)
- **Delivery**: presigned S3 URLs minted by the API for consent-passed datasets only.

4.3 Secrets, identity, observability

- **Secrets Manager** for DB creds, third-party API keys — no hardcoded creds anywhere (this is also a project safety norm).
 - **IAM roles per component** with least privilege; the consent boundary in §2 is enforced by role scoping.
 - **CloudWatch** logs + metrics; pipeline provenance and job status live in Postgres `jobs`.
 - **KMS** CMK for S3 + RDS encryption.
-

5. Data flow (end to end)

1. Capture uploaded → `actuate-raw` (encrypted, restricted). Row in `episodes` (status=ingested), `consent` recorded.
 2. Step Functions runs perceive→fuse→canonical (GPU+CPU Batch) → artifacts to `actuate-work/canonical/<id>/vN`, `jobs` updated per stage.
 3. `certify` runs quality + LLM-judge + strategy-alignment + **fail-closed consent/PII gate**; writes `certifications`. Only `pii_status=passed` proceeds.
 4. `retarget` (per target embodiment) → `retarget_results`, sim-validation status.
 5. `package` (packaging role only) → LeRobot v3 / RLDS to `actuate-delivery/<customer>/<dataset>/<version>`; `datasets` row + manifest.
 6. API mints presigned URLs; customer pulls. Un-consented data is unreachable by both code and IAM.
-

6. Infrastructure-as-Code

AWS CDK in Python (matches the codebase language; one repo `infra/` app). Stacks: `StorageStack` (buckets + KMS + lifecycle + policies), `DataStack` (RDS/Aurora + Secrets + Glue/Athena), `ComputeStack` (Batch envs + Step Functions + ECR), `ServiceStack` (Fargate/App Runner + ALB + CloudFront). Two environments (`dev`, `prod`) via CDK context. (Terraform is a fine alternative if you prefer HCL; CDK-Python keeps everything in one language.)

7. Cost pragmatics (with credits)

Spot GPU for Batch; Aurora Serverless v2 min-capacity or a small RDS instance you stop when idle; Fargate for spiky serving; S3 Intelligent-Tiering + Glacier lifecycle on raw video; never run GPU 24/7; delete churning intermediate `work/` artifacts on a lifecycle rule once canonical is finalized. Video egress to customers is a real cost — prefer presigned URLs over re-hosting.

8. Honest status

None of this is provisioned. It is the target architecture. It is deliberately buildable in the order: **S3 + KMS + IAM (consent boundary)** → **RDS/Postgres catalog** → **containerize `actuate`** → **single-box or Batch processing** → **Fargate API** → **dashboard**. The consent boundary (IAM + bucket policy + fail-closed gate) should be stood up and tested *first*, before any real capture data lands, and re-verified whenever roles change — the same discipline the built consent gate already follows in code.